

A Machine-Learning Benchmark for Semileptonic VBF Higgs-Pair Events in a Coupling Scan

Felix Förster, Lars Schneider, and Johannes Mesner

Technical University of Munich
Advisors: Lars Linden and Celine Stauch
Ludwig Maximilian University of Munich

Abstract. We present a machine-learning benchmark for non-resonant vector-boson-fusion Higgs-pair production in the semileptonic channel $HH \rightarrow bbWW \rightarrow bbqq\ell\nu$. From truth-level Monte Carlo samples, we build a selected and matched event-level dataset and represent each event with kinematic features for the main physics objects and their combinations. We then study whether this representation can distinguish different values of the coupling scan parameter cvv , with focus on the binary separation of the Standard Model reference point $\text{cvv} = 1$ from the remaining sampled classes. We find that this task is learnable with the available feature set. The full representation performs best overall, while several nonlinear models reach similar accuracy. In contrast, scaling and oversampling show mixed effects, and ensemble methods do not clearly improve on the best individual models. These results show that the matched event representation retains useful information for coupling classification in semileptonic VBF Higgs-pair events.

1 Introduction and Objective

Vector-boson-fusion (VBF) Higgs-pair production is a direct probe of the Higgs interaction with electroweak gauge bosons. The coupling of vector bosons to Higgs boson pairs is predicted by the Standard Model but has not yet been observed experimentally [7]. While the VBF channel is particularly sensitive to this coupling, the very small production cross section severely limits the available event statistics. At the same time, the final state is challenging to reconstruct. In this project, we study the semileptonic channel $HH \rightarrow bbWW \rightarrow bbqq\ell\nu$ together with the two VBF quarks. Even at truth level, this topology contains several correlated objects, boosted composite systems, and nontrivial matching relations between decay products and jets. Because of this, machine-learning methods that can reliably distinguish different coupling hypotheses from limited data could play an important role in future experimental searches for this process.

The central parameter of this study is the coupling scan variable cvv , which we use here as the project notation for κ_{2V} . The point $\text{cvv} = 1$ corresponds to the Standard Model reference sample. The main physics question is whether the event representation produced by the dataset pipeline contains enough information to distinguish this Standard Model point from samples generated at other coupling values. This is relevant because VBF Higgs-pair production is sensitive to changes in the Higgs–vector-boson interaction. At the same time, the matched event sample provides a controlled setup in which this sensitivity can be studied systematically.

The machine-learning part of the project is motivated by the structure of the dataset. After event selection and truth-to-jet matching, each event is described by a set of object-level and derived kinematic features. These features encode the VBF quarks, the hadronic and leptonic W systems, the $H \rightarrow b\bar{b}$ candidate, and combinations of these objects. The main benchmark task is then to separate C_1 ($\text{cvv} = 1$) from $C_{\text{not } 1}$ ($\text{cvv} \neq 1$). The full set of cvv classes is still useful because it gives additional context for the structure of the dataset and the coupling

dependence of the problem.

This report documents the full workflow in a coherent way. It describes how the matched dataset is constructed, how the event representation is defined, and what the current benchmark results imply for the learnability of the classification task. The main contribution of the report is therefore not a new theoretical result. Instead, it is a clear description of the dataset, the benchmark setup, and the conclusions that can already be drawn from the current study.

2 Dataset Construction and Physics Context

2.1 Selected Event Topology

The dataset is built for non-resonant vector-boson-fusion Higgs-pair production in the semileptonic channel. In the topology studied here, one Higgs boson decays to $b\bar{b}$, while the second decays to WW with one hadronic and one leptonic W decay. Together with the two VBF quarks, this gives the final state

$$HH \rightarrow b\bar{b}WW \rightarrow b\bar{b}q\bar{q}\ell\nu. \quad (1)$$

This channel offers a good balance between a rich event structure and a clear mapping to the feature representation used in the later machine-learning study. The VBF quarks define the tagging side of the event, the hadronic and leptonic W branches encode the $H \rightarrow WW$ decay, and the two b quarks define the $H \rightarrow b\bar{b}$ system. In the boosted regime, the hadronic W and $H \rightarrow b\bar{b}$ systems can become sufficiently collimated that reconstruction based on large-radius jets is well motivated.

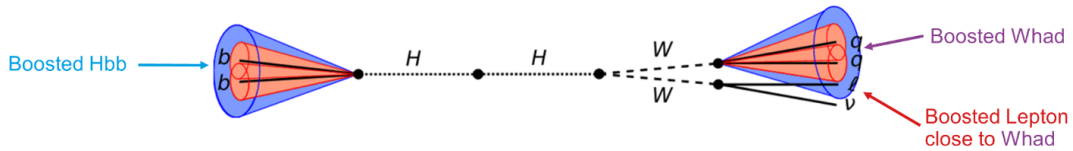


Fig. 1. Schematic view of the boosted semileptonic VBF Higgs-pair topology considered in this work. One Higgs boson decays to $b\bar{b}$, while the second decays to WW with one hadronic and one leptonic branch. The boosted $H \rightarrow b\bar{b}$ and hadronic W systems motivate the use of large-radius jet matching in the dataset construction.

2.2 Dataset Pipeline

The machine-learning dataset used in the benchmark is constructed from truth-level Monte Carlo samples generated at several values of the coupling parameter cvv , which in this project denotes the scan over κ_{2V} . The point $\text{cvv} = 1$ defines the Standard Model reference sample. Rather than using the full event record directly, the dataset is restricted to the semileptonic VBF Higgs-pair topology described above. Only events for which the relevant truth-level objects can be matched consistently to jets are kept.

The matching procedure is performed in two stages. First, the hadronic W system and the $H \rightarrow b\bar{b}$ system are matched to large-radius jets within $\Delta R < 1.0$. The two VBF quarks are then matched to the remaining jets within $\Delta R < 0.4$. Jets are treated as large-radius candidates

if they satisfy $p_T > 200$ GeV and $|\eta| < 2$. This ordering ensures that the large-radius jets are assigned to the boosted composite systems before the VBF-side matching is attempted. The final dataset is therefore not a raw generator sample, but a selected and successfully matched event table.

2.3 Matched Sample and Class Imbalance

The selected dataset is strongly non-uniform across coupling points. The numbers of generated and matched events are summarized in Table 1. The most striking feature is the strong suppression of the $\text{cvv} = 1.0$ sample after selection and matching. Although this sample starts with the largest number of generated events, it contributes by far the smallest number of matched events to the final dataset. In particular, the kept fraction drops to 0.4% at $\text{cvv} = 1.0$, compared to $\mathcal{O}(10\text{--}20\%)$ for the other coupling points.

Table 1. Generated and matched event counts used to construct the machine-learning dataset.

cvv	Generated	Matched	Kept fraction
0.0	2,000,000	243,917	12.20%
0.5	2,000,000	180,456	9.02%
1.0	5,000,000	19,796	0.40%
1.5	1,000,000	209,702	20.97%
2.0	1,000,000	204,740	20.47%
3.0	1,000,000	185,928	18.59%

This imbalance plays a central role in the design of the later benchmark. It motivates the use of balancing strategies and gives the binary classification task around the Standard Model point clear practical relevance. In particular, the separation of C_1 from $C_{\text{not } 1}$ is not only physically motivated, but also statistically nontrivial, since the reference sample is strongly depleted after dataset construction. We do not attempt to explain the origin of this depletion here. For the purposes of this report, it is treated as an empirical property of the selected dataset.

3 Event Representation and Task Definition

3.1 Event Representation

Each selected event is represented by a fixed-size feature vector. This representation combines truth-level kinematics, jet-level information, matching distances, and derived variables built from multiple properties. The machine-learning input is therefore not just an arbitrary list of features. It is designed to stay closely connected to the physical structure of the event.

Table 2 summarizes the main feature groups written by the event builder. The VBF block describes the two forward quark candidates and their associated jets. The hadronic W and $H \rightarrow b\bar{b}$ blocks each store both the composite truth object and its large-radius jet representation, together with the corresponding child particles. The leptonic block stores the charged lepton and neutrino from the leptonic W decay. In addition, the dataset includes derived variables such as invariant masses and transverse momenta of combined systems, which capture broader event structure.

The stored variables follow standard collider kinematics. In particular, the representation includes transverse momentum p_T , pseudorapidity η , azimuthal angle ϕ , invariant mass, and angular distances defined through ΔR . Together, these quantities describe both the individual objects and their kinematic and geometric relations inside the full event.

Table 2. Main feature groups in the event-level dataset.

Block	Content
VBF quarks	Truth features for <code>vbf_q1</code> and <code>vbf_q2</code> , jet features, and ΔR values describing the VBF tagging side of the event
Hadronic W	Truth features for <code>w_qq</code> , jet features, daughter-quark features, and <code>w_qq_dR</code>
Leptonic W	Features for the charged lepton <code>w_lep_l</code> and neutrino <code>w_lep_v</code> from the leptonic decay
$H \rightarrow b\bar{b}$	Truth features for <code>h_bb</code> , jet features, daughter b -quark features, and <code>h_bb_dR</code>
Combination observables	Variables such as <code>hbb_wqq_m</code> , <code>hbb_lep_m</code> , <code>wqq_lep_m</code> , and <code>q1_q2_m</code> , together with related kinematic quantities for combined systems

3.2 Structure in Feature Space

The usefulness of this representation becomes clearer in the dataset analysis plots. The correlation analysis in Appendix A shows the internal correlation structure of the exported variables after sorting them by kinematic type. The strongest patterns appear within feature groups that belong to the same physical system, for example the $H \rightarrow b\bar{b}$ block, the hadronic W block, and the VBF side of the event. This is exactly what one would expect from a representation that follows the event topology instead of discarding it.

It is also useful to look at individual feature distributions for C_1 and $C_{\text{not } 1}$ directly. Figure 2 shows two representative examples. The energy of the leading VBF jet, `vbf_q1_jet_e` (left), produces nearly identical distributions for both classes. The shapes peak in the same region and fall off at similar rates, so this variable alone offers very little separation. In contrast, the pseudorapidity of the second VBF jet, `vbf_q2_jet_eta` (right), shows a visible shape difference between the two classes. The $C_{\text{not } 1}$ distribution is more sharply peaked in the forward region, while C_1 events are spread more broadly toward central pseudorapidities.

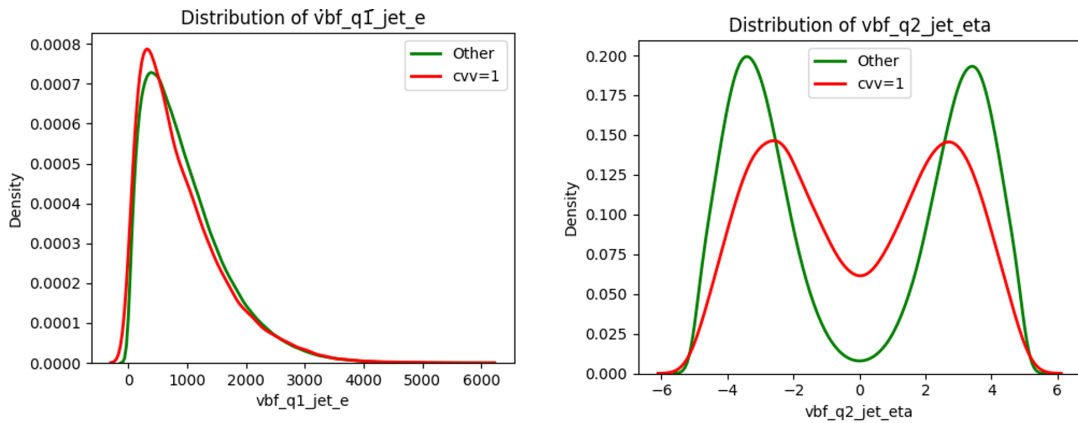


Fig. 2. Two representative feature distributions comparing C_1 ($\text{cvv}=1$, red) and $C_{\text{not } 1}$ (green). **Left:** The energy of the leading VBF jet (`vbf_q1_jet_e`) shows nearly complete overlap between the two classes. **Right:** The pseudorapidity of the second VBF jet (`vbf_q2_jet_eta`) shows a clear shape difference, with C_1 populating a broader, more central range.

This pattern holds across the full feature set. Most variables show strong overlap between the two classes, which means no single feature can drive the classification on its own. However, a subset of features, especially angular variables on the VBF side, do carry measurable differences. This supports the binary framing: the separation between C_1 and $C_{\text{not } 1}$ is possible, but the signal is spread thinly across many variables rather than concentrated in one. The complete set of feature distributions is collected in Appendix E.

For the current report, the most relevant analysis is the binary task distinguishing C_1 and $C_{\text{not } 1}$. Figure 3 shows the strongest feature correlations with this binary target. Informative variables appear in several parts of the event at once, including the VBF system, the $H \rightarrow b\bar{b}$ candidate, the matching variables, and the derived combination observables. This supports the choice to use both detailed kinematic inputs and a smaller set of summary variables in the benchmark.

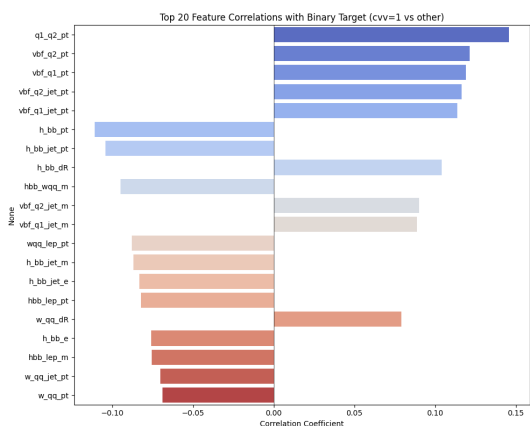


Fig. 3. Top feature correlations with the binary target C_1 versus $C_{\text{not } 1}$. The visible signal comes from several feature groups rather than from a single isolated observable. A simplified interpretation of the values is: 0.2 = *bad*, 0.5 = *ok*, 0.8 = *good*, 1.0 = *perfect correlation*.

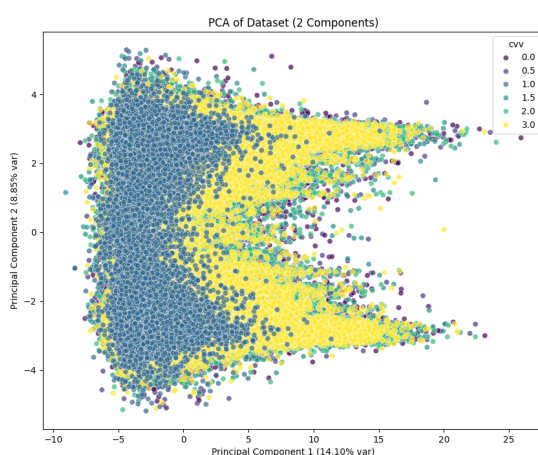


Fig. 4. PCA projection of the full dataset with raw cvv labels. The strong overlap between classes illustrates why the multiclass setting is more challenging than the binary framing used in the main benchmark.

The multiclass structure is visibly weaker. Figure 4 shows a two-component PCA projection of the full dataset with the raw cvv labels. The classes overlap strongly in this low-dimensional projection, even though some broad structure remains visible. This does not mean that multiclass classification is impossible. It does, however, match the main benchmark story of this report: the binary separation around $cvv = 1$ is the clearer and better supported central task.

3.3 Target Labels and Task Definitions

The default target column in the classification framework is cvv . The data loader can use the raw labels directly, remove selected labels, or merge several labels into larger groups. The same dataset therefore supports both a multiclass setup over the visible cvv values and reduced or merged task definitions.

To keep the notation compact, we write C_v for the class of events with label cvv equal to v , and $C_{\text{not } v}$ for the complementary class obtained by merging all other labels. This notation matches the binary benchmark framing used throughout the current work. In particular, the available results focus on the separation of C_1 and $C_{\text{not } 1}$. While the dataset supports C_v with

$v \in [0.0, 0.5, 1.0, 1.5, 2.0, 3.0]$ the presented benchmark focuses on the classification of C_1 and $C_{\text{not } 1}$. Refer to the Appendix C for further clarification behind this decision.

This task definition also fits the observed dataset structure. The strong class imbalance around $\text{cvv} = 1$ makes the binary problem more than a simple regrouping of labels. It turns the benchmark into a test of whether the event representation contains enough information to isolate the sparse Standard Model sample from the rest of the events. For that reason, the binary framing is not only convenient, but also physically and statistically well motivated for the presented study.

4 Method

4.1 Machine Learning Models

In this section, we describe some fundamental machine learning techniques of our analysis. We begin with the optimization algorithm that underlies many learning methods, followed by progressively more complex models ranging from linear classifiers to ensemble methods and neural networks. Each technique offers distinct advantages and trade-offs in terms of interpretability, computational efficiency, and predictive power. Understanding these methods is essential for selecting appropriate approaches for the binary classification task between C_1 and $C_{\text{not } 1}$, where the distinction between signal events and background processes requires robust pattern recognition. For every model the goal will be to predict the correct class y from a given input vector x , while our model prediction is $\hat{y}$.

Stochastic Gradient Descent Stochastic Gradient Descent (SGD) is an optimization algorithm that minimizes a loss function by iteratively updating model parameters in the direction of the negative gradient [1]. Unlike batch gradient descent, which computes the gradient using the entire dataset at each iteration, SGD estimates the gradient using a single training example or a small batch of examples. This stochastic nature introduces noise into the optimization process, which can help escape local minima but also results in more erratic convergence behavior.

The SGD update rule for model parameters θ with learning rate η and loss function $\mathcal{L}$ is given by:

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t; x_i, y_i) \quad (2)$$

where (x_i, y_i) is a training sample or batch. The learning rate is a critical hyperparameter that controls the step size of each update; too large values cause divergence while too small values lead to slow convergence. Learning rate schedules, which gradually decrease the learning rate during training, are commonly employed to balance initial rapid progress with fine-grained final convergence. Extensions such as momentum, which accumulates past gradients to smooth updates, further improve convergence stability. SGD serves as the optimization backbone for many machine learning models, including logistic regression and neural networks, making it fundamental to understanding modern ML practices.

There are also more advanced optimizers with the most prominent being Adam. These include more functionality to escape local minima and increasing the convergence speed. For the sake of simplicity we will stick to SGD and assume its the backbone for all our methods including the deep learning methods, multilayer perceptron (MLP) and DeepSet later on.

Logistic Regression Logistic Regression is a linear classification algorithm that models the probability of a binary outcome as a function of input features [4]. Despite its name, it is used for classification rather than regression. The algorithm applies the logistic (sigmoid) function to a linear combination of input features to produce probabilities in the range $[0, 1]$.

For a feature vector x with weights w and bias b , the predicted probability is:

$$P(y = 1|x) = \sigma(w^T x + b) = \frac{1}{1 + \exp(-(w^T x + b))} \quad (3)$$

The model is trained by maximizing the likelihood $\mathcal{L}$ of the training data, equivalent to minimizing the binary cross-entropy loss using SGD (which is the loss also used for the deep learning methods):

$$\mathcal{L}_{BCE}(w, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4)$$

Logistic regression provides interpretable results through feature weights, making it valuable for understanding which input variables drive predictions. While its linear decision boundary limits its ability to capture complex nonlinear relationships, it offers computational efficiency and serves as a useful benchmark against more sophisticated models.

Decision Tree A Decision Tree is a hierarchical model that makes predictions by recursively partitioning the feature space into regions based on simple decision rules [2]. The tree structure consists of internal nodes (representing tests on specific features), branches (representing the outcomes of these tests), and leaf nodes (representing final predictions). At each internal node, the algorithm selects a feature and a split point that best separates the training examples according to the target variable, measured by an impurity metric such as Gini impurity or information gain (entropy).

For a binary classification task with class proportions p_1 and $p_2 = 1 - p_1$ in a node, Gini impurity G is defined as:

$$G = 1 - \sum_{k=1}^2 p_k^2 \quad (5)$$

A pure node (all examples belong to one class) has Gini impurity of 0, while a node with equal class proportions has maximum impurity of 0.5. The algorithm selects the split that maximizes the weighted reduction in impurity across the resulting child nodes. This splitting process continues recursively until stopping criteria are met, such as a maximum tree depth, minimum number of samples per leaf, or when further splits do not yield significant impurity reduction.

During prediction, a new example traverses the tree from the root to a leaf node by following the decision rules at each internal node. The leaf node's class distribution determines the final prediction, typically by majority voting. Decision trees are interpretable, handle both numerical and categorical features, and can capture nonlinear relationships and feature interactions. However, individual trees are prone to overfitting and typically exhibit high variance, which motivates ensemble methods like Random Forests or Gradient Boosting.

Random Forest Random Forest is an ensemble learning method that constructs multiple decision trees during training and aggregates their predictions through voting or averaging [3].

Each tree is trained on a bootstrap sample of the data (sampling with replacement) and considers only a random subset of features at each split. This combination of bootstrap sampling and feature randomization introduces diversity among the trees, which is key to the method’s success.

The final prediction for classification is determined by majority voting across all trees:

$$\hat{y} = \text{mode}\{h_t(x)\}_{t=1}^T \quad (6)$$

where $h_t(x)$ is the prediction of the t -th tree and T is the total number of trees. This ensemble approach reduces overfitting compared to individual decision trees, as errors from individual trees tend to cancel out when predictions are averaged. Random forests naturally handle high-dimensional data, are robust to outliers, and provide feature importance measures through the reduction in impurity achieved by each feature across all trees.

The method’s ability to capture nonlinear relationships and interactions between features makes it well-suited for particle physics applications where kinematic variables may exhibit complex correlations. Key hyperparameter include the number of trees, the maximum depth of trees, and the number of features considered at each split.

Gradient Boosting Gradient Boosting is an ensemble technique that builds models sequentially (often decision trees), with each new model correcting the errors of the previous ones [5]. Unlike random forests, which train trees independently and aggregate predictions, gradient boosting trains trees in a staged manner, with each tree fitting the negative gradient (similar to SGD) of the loss function (the pseudo-residuals).

The gradient boosting model is built additively:

$$F_m(x) = F_{m-1}(x) + h_m(x) \quad (7)$$

where F_{m-1} is the current model, h_m is a weak learner (typically a shallow decision tree), and the tree is fitted to approximate the negative gradient:

$$h_m \approx -\frac{\partial \mathcal{L}(y, F(x))}{\partial F(x)} \quad (8)$$

This gradient descent in function space allows gradient boosting to optimize arbitrary differentiable loss functions. The algorithm typically employs a learning rate (also called shrinkage or step size) to prevent overfitting by scaling each tree’s contribution:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x), \quad 0 < \nu \leq 1 \quad (9)$$

Smaller learning rates require more trees but often yield better generalization. Other important hyperparameter control tree complexity, such as maximum depth, minimum samples per leaf, and subsampling fraction. Gradient boosting methods have achieved state-of-the-art performance on tabular data.

Histogram-based Gradient Boosting Histogram-based Gradient Boosting is a computational enhancement to traditional gradient boosting that significantly improves training speed without sacrificing predictive accuracy [8]. Instead of finding optimal split points across all unique feature values, histogram-based methods bin continuous features into a fixed number of discrete bins (typically 256). This transforms the split-finding problem from $O(n)$ per feature to $O(\text{bins})$, dramatically reducing computational complexity while approximately preserving the

optimal splits.

The binning operation is performed once at the beginning of each boosting iteration, and all subsequent computations operate on the binned data. Despite the discretization, the performance degradation is minimal because the bins are chosen to capture the distribution of feature values adaptively.

Multi Layer Perceptron The Multi Layer Perceptron (MLP) is a fundamental neural network architecture consisting of layers of interconnected neurons that process information through weighted connections [6]. An MLP comprises an input layer that receives feature values, one or more hidden layers that learn hierarchical representations, and an output layer that produces predictions. Each neuron applies a linear transformation followed by a nonlinear activation function, enabling the network to learn complex nonlinear relationships.

The forward propagation through a single hidden layer and output layer is computed as:

$$h = \sigma(W^{(1)}x + b^{(1)}), \quad \hat{y} = W^{(2)}h + b^{(2)} \quad (10)$$

where $W^{(1)}, W^{(2)}$ are weight matrices, $b^{(1)}, b^{(2)}$ are bias vectors, and σ is a nonlinear activation function such as ReLU, sigmoid, or tanh. The universal approximation theorem states that a feedforward network with a single hidden layer can approximate any continuous function given sufficient neurons, though in practice deeper networks often require fewer parameters and generalize better.

The smallest form of a MLP, with just an input and output layer is the same as Logistic Regression when choosing sigmoid as the nonlinear activation function.

Training MLPs involves backpropagation, which computes gradients of the loss with respect to all weights using the chain rule. These gradients are then used to update weights via an optimizer such as SGD or Adam. Key design choices include the number of hidden layers, neurons per layer, activation functions, and regularization techniques such as dropout and weight decay. MLPs serve as the foundation for more advanced neural architectures and remain effective for structured tabular data when properly regularized.

4.2 DeepSets

DeepSets [10] is a neural network architecture designed to process unordered sets of elements while maintaining permutation invariance. This property makes DeepSets particularly well-suited for our binary classification, where collision events can be treated as sets that need to be classified without being dependent on a particular ordering.

In the context of this study, we treat multiple events as a set input to the DeepSets model. Rather than classifying a single event in isolation, the model receives a set of n events and learns to produce a classification based on the entire set. This approach allows the model to leverage statistical patterns across multiple events, where the information of multiple events provides better discrimination between C_1 and $C_{\text{not } 1}$ events. Traditional neural networks require fixed-size inputs with a defined order, which is problematic when we want to process varying numbers of events together. DeepSets addresses this challenge by applying the same transformation to each event independently and then aggregating all transformed events into a fixed-size representation. Note that only the DeepSet architecture uses multiple events for classification at a time. All other methods presented in this study only rely on a single sample per prediction.

The mathematical foundation of DeepSets relies on the Universal Approximation Theorem for Sets, which states that any permutation-invariant function on a set can be approximated by a neural network of the form:

$$F(\{x_1, x_2, \dots, x_n\}) = \rho \left(\sum_{i=1}^n \phi(x_i) \right) \quad (11)$$

where ϕ is a per-element transformation (typically a neural network), ρ is a set-level aggregation function and n is the number of input features. This decomposition ensures that the output remains invariant to the ordering of input elements, as the sum (or other permutation-invariant operation) treats all elements equally regardless of their position.

In our implementation, the per-element encoder ϕ processes each event by first encoding the individual particles and jets within that event. Within this per-event processing, we apply self-attention to allow the different particles and jets within an event to attend to each other and learn pairwise interactions. Using more self-attention layers will increase the reach of the attend that is possible to learn. The encoded events are then aggregated using a permutation-invariant function to produce a fixed-size set-level embedding. Finally, a classification head processes this embedding to produce the desired output (binary classification or multi-class classification).

Attention Mechanism Attention [9] is a mechanism that allows neural networks to dynamically focus on the most relevant parts of the input when producing an output. Originally developed for sequence-to-sequence models in natural language processing, attention has become a fundamental building block in modern deep learning.

To get an intuition what attention provides, you can look at a sample sentence like: “The red car drives behind the blue ball.”. Here the word “red” closely is connected to the word “car” but not to the word “ball”. Of course this will change for every combination of words since attention scales quadratically and is a pairwise value.

The attention mechanism computes a weighted sum of values, where the weights are determined by a compatibility function between a query and the corresponding keys. Mathematically, scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (12)$$

where Q (queries), K (keys), and V (values) are learned linear projections of the input, and d_k is the dimensionality of the key vectors. The softmax function produces a probability distribution over the input positions, determining how much each element should contribute to the output.

Self-Attention in DeepSets Self-attention is a variant of attention where all queries, keys, and values come from the same source. In our implementation, we apply self-attention within the ϕ function to process the particles and jets within each event. Each particle or jet feature attends to all other particles and jets features in the same event, allowing pairwise interactions to be captured. This enables the model to learn correlations between different objects in the same event, such as kinematic relationships between decay products.

The key idea is that self-attention allows information to flow between all pairs of elements within an event. With multiple self-attention layers, elements can indirectly influence each other

through intermediate elements, effectively learning higher-order interactions. For example, a particle can attend to another particle that attends to a third particle, enabling the model to capture complex multi-body relationships in the event, we have used three self attention layers for our experiments.

Attention pooling Attention pooling is a permutation-invariant aggregation method that computes a weighted sum of the encoded elements, where the weights are learned through an attention mechanism. Unlike self-attention which captures pairwise interactions between all elements, attention pooling uses a single learned query vector to compute attention weights for all elements simultaneously.

Given a set of encoded elements $\{h_1, h_2, \dots, h_n\}$, attention pooling computes:

$$a_i = \frac{\exp(q \cdot h_i)}{\sum_{j=1}^n \exp(q \cdot h_j)} \quad (13)$$

$$\text{AttentionPool}(h_1, \dots, h_n) = \sum_{i=1}^n a_i h_i \quad (14)$$

where q is a learned query vector and a_i are the attention weights that sum to one. This produces a weighted average of the elements, where more informative elements receive higher weights. The key difference from self-attention aggregation is that attention pooling does not compute pairwise interactions between elements—instead, it assigns each element a relevance score relative to the fixed query, making it computationally more efficient while still learning which elements are most important for the classification task.

4.3 Data Processing and Transformation Pipeline

To ensure robust model convergence and mitigate biases stemming from class imbalances or feature scaling disparities, we implement a rigorous, modular data processing pipeline. This section details the data normalization techniques, the dynamic over- and undersampling strategies, and additional dataset refinement features.

Data Normalization and Scaling Data precision is a vital point when working with very detailed decimal numbers. The collected dataset is in 64-bit floating-point precision (`float64`), meaning we use 64 bit to save one decimal number. While this is extremely accurate, it is slow to calculate with and expensive to store in memory. To optimize memory footprint and computational efficiency during training, all features are inherently cast to 32-bit floating-point precision (`float32`). This is the gold standard in ML and is also accurate enough to suit our particle physics application.

Feature scaling is critical for distance-based and gradient-descent optimized algorithms. This includes some but not all of our model architectures. Considering this, we apply a transformation to the data before training or evaluating on it. This transformation depends heavily on the technique used but aims in general to make the data “more understandable” to the ML architectures. Transformations often include offsetting and rescaling the data. If we disregard this normalization step the calculated loss is heavily biased towards features with higher value (e.g. `pt` or `e`) and will completely ignore others (e.g. `phi` or `eta`).

The framework supports six distinct data scaling strategies:

- **Standardization:** Utilizes zero-mean, unit-variance scaling. Each feature x_i is transformed via $x'_i = \frac{x_i - \mu_i}{\sigma_i}$, where μ_i and σ_i are the mean and standard deviation of the feature in the training set. Standardization is the common scaling technique and fits most application areas especially in Deep Learning.
- **Min-Max Scaling:** Linearly compresses features into an $[0, 1]$ interval. This is done via $x'_i = \frac{x_i - x_{min}}{x_{max} - x_{min}}$, where x_{min} and x_{max} are the maximum and minimum feature value in the training set respectively.
- **Max-Absolute Scaling:** Scales features by their maximum absolute value, bounding them in the $[-1, 1]$ range without shifting the distribution center, thereby preserving sparsity in the data. This feature transformation of x_i is done through applying: $x'_i = \frac{x_i}{\max(|x|)}$
- **Robust Scaling:** Employs statistics that are robust to outliers, subtracting the median and scaling the data according to the Interquartile Range (IQR). The feature x_i is scaled by $x'_i = \frac{x_i - \text{median}(X)}{Q_3(X) - Q_1(X)}$, where Q_3 and Q_1 is the 75th and 25th percentile respectively.
- **Yeo-Johnson Power Transformation:** Applies a monotonic power transformation to stabilize variance and minimize skewness, transforming the feature distribution to be more Gaussian-like. Unlike the Box-Cox transform, Yeo-Johnson inherently supports negative values.

$$\psi(x_i, \lambda) = \begin{cases} \frac{(x_i+1)^\lambda - 1}{\lambda} & \text{if } x_i \geq 0, \lambda \neq 0 \\ \ln(x_i + 1) & \text{if } x_i \geq 0, \lambda = 0 \\ -\frac{(-x_i+1)^{2-\lambda} - 1}{2-\lambda} & \text{if } x_i < 0, \lambda \neq 2 \\ -\ln(-x_i + 1) & \text{if } x_i < 0, \lambda = 2 \end{cases}$$

where λ is the transformation parameter and is usually determined through a Maximum Likelihood Estimation.

- **Quantile Transformation:** Non-linearly maps the data to precisely match either a uniform or a normal distribution, relying on the empirical cumulative distribution function. Depending on the target distribution we map the input x_i by $x'_i = G^{-1}(F(x_i))$, where F is the Empirical Cumulative Distribution Function and G^{-1} is the Quantile Function of the target distribution.

In machine learning and especially physics, sometimes the absolute magnitudes of measured variables are highly volatile due to energy scaling or intensity, whereas the relative relationship between the variables within the same event or two different physical particles is the actual distinguishing characteristic separating classes. Sample-wise normalization removes the absolute magnitude completely and forces the model to learn purely from the internal geometry/proportions of the features within a single event. For this we implemented sample-wise normalizations using vector norms. Depending on the configuration, individual event vectors are scaled to unit norm based on the L_1 , L_2 , or maximum (max) norm, effectively adjusting feature magnitudes relative to the specific sample.

Dynamic Over- and Undersampling Event classification in particle physics often suffers from heavily imbalanced target distributions. This also applies to our collected dataset as you can see in Table 1. To counteract the algorithmic bias towards majority classes ($C_{\text{not } 1}$ in our case), runtime data balancing is enforced via a custom data balancer module. In simple words, we want to show the models the same amount of each class because otherwise it will bias towards the majority class. But at the same time, we want to maximize the amount of information the models see while training. We implemented three primary strategies to balance the training data:

1. **Undersampling:** The target number of instances per class is constrained to the size of the smallest minority class C_1 , $N_{\text{target}} = \min_c(N_c) = 19.796$, where N_c is the cardinality of class c . All classes strictly larger than this target are subsampled uniformly at random.

2. **Oversampling (Unbounded):** Minority classes are duplicated randomly with replacement until they match the size of the largest majority class, $N_{\text{target}} = \max_c(N_c) = 1.024.743$. This means that the models see a sample of C_1 around 50 times while encountering samples of $C_{\text{not } 1}$ only once.
3. **Factorized Oversampling:** To prevent excessive duplication of heavily underrepresented minority classes which can lead to severe model overfitting, an oversampling factor f_{os} can be employed. In this regime, the target size is defined relative to the smallest minority class size:

$$N_{\text{target}} = \min_c(N_c) \times f_{\text{os}} \quad (15)$$

Consequently, any class inherently smaller than N_{target} is randomly oversampled, while classes exceeding N_{target} are undersampled without replacement. This hybrid approach allows fine-grained control over the data distribution spread, bridging the gap between severe undersampling data-loss and aggressive oversampling redundancy. In our case, we chose $f_{\text{os}} = 3$ because it is a common ground between redundancy and still providing additional information on the $C_{\text{not } 1}$ class.

4.4 Framework

To systematically evaluate and contrast multiple machine learning algorithms under precisely controlled conditions, we developed a custom, highly modular Python-based experimental framework. The architecture centers around a dynamic **Registry** system and a central **ExperimentRunner**, entirely decoupled from hard-coded execution logic. All experimental parameters, including datasources, preprocessing configurations, and the specific subsets of deployed modeling methods are defined globally via structured JSON configuration files. This separation allows for fully reproducible, version-controlled execution of extensive ablation studies and hyperparameter grid searches without necessitating structural code modifications. You can find an example and explanation of the configuration file in the Appendix B. This means we can run complex experiments containing many models, norms, datasets and scaling methods through configuring a single JSON file. This setup is inspired by a more complex and compute cluster conform version, used for example in the OpenClip project.

The execution pipeline is rigorously automated. Once an experiment is initialized, the **ExperimentRunner** parses the combinatorial permutations of specified datasources and preprocessing pipelines, systematically evaluating every registered classification method against each distinct state of the dataset. To ensure robust scalability and mitigate prolonged execution times during massive grid searches, the framework supports multiprocessing both at the meta-level and the method level. This means that we are able to train multiple models at the same time while also being capable of training an individual method on multiple compute cores. This flexibility is especially useful when running large scale experiments with 40+ trainable combinations or for methods that are intensive in training like Histogram Gradient Boosting or XGBoost.

For every executed combination, the framework autonomously computes and logs comprehensive validation metrics (e.g., accuracy, macroscopic F_1 -scores, precision-recall thresholds), subsequently aggregating these results into centralized analytical outputs. Beyond raw numerical telemetry, the system automatically exports unified artifact manifestations—such as serialized binary model states (.pkl/.onnx for cross-platform portability), graphical confusion matrices, misclassification overlaps, and cross-method agreement matrices—directly into strictly versioned output directories corresponding to the specific experimental topology. Or in other words: The system logs and plots everything there is automatically and saves models for later usage. This makes rerunning experiments after changes simple.

5 Evaluation

This section gives a detailed explanation of the experiment setup and results. In addition, we provide extensive ablation studies to motivate our choices for the mechanics introduced before.

5.1 Setup

Data The primary ablation studies are conducted on the large dataset we collected (see section 2. We split this collection into 80% for training and 20% for validation. Given the nature of our project, which focuses on prototyping possible classification solutions and considering the limited amount of data, we did not create an additional test dataset. If not specified otherwise, we use three times oversampling for the minority class C_1 , as motivated previously in section 4.3. Note that we do not use oversampling for the validation dataset; instead, we balance it evenly across classes.

Metrics We use prediction accuracy as our primary evaluation metric. Given the constellation of our validation set, accuracy is a reasonable choice since there is no bias in the data. We also analyzed the corresponding F1 scores, but they are for all results extremely similar and not included in the following metrics.

5.2 Data Scaling

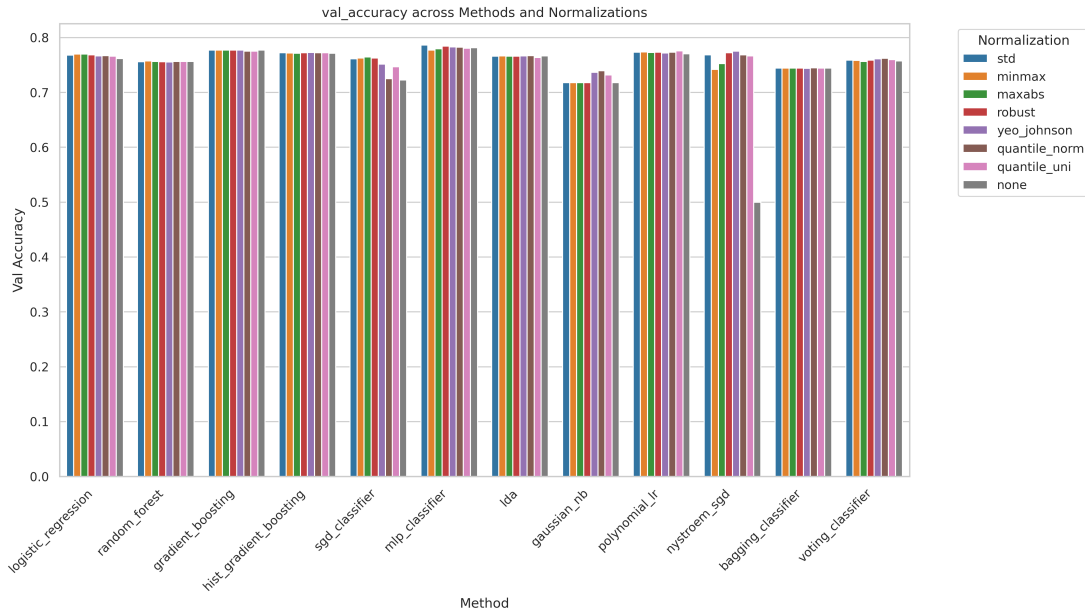


Fig. 5. This figure shows 8 different scaling approaches on 12 ML algorithms. Depending on the model architecture, scaling brings some minor improvements while on other there is almost no difference to the baseline with no scaling.

As mentioned in 4.3, data scaling is a common approach to improve model performance in ML. Depending on the Algorithm, different data rescaling methods perform better than others.

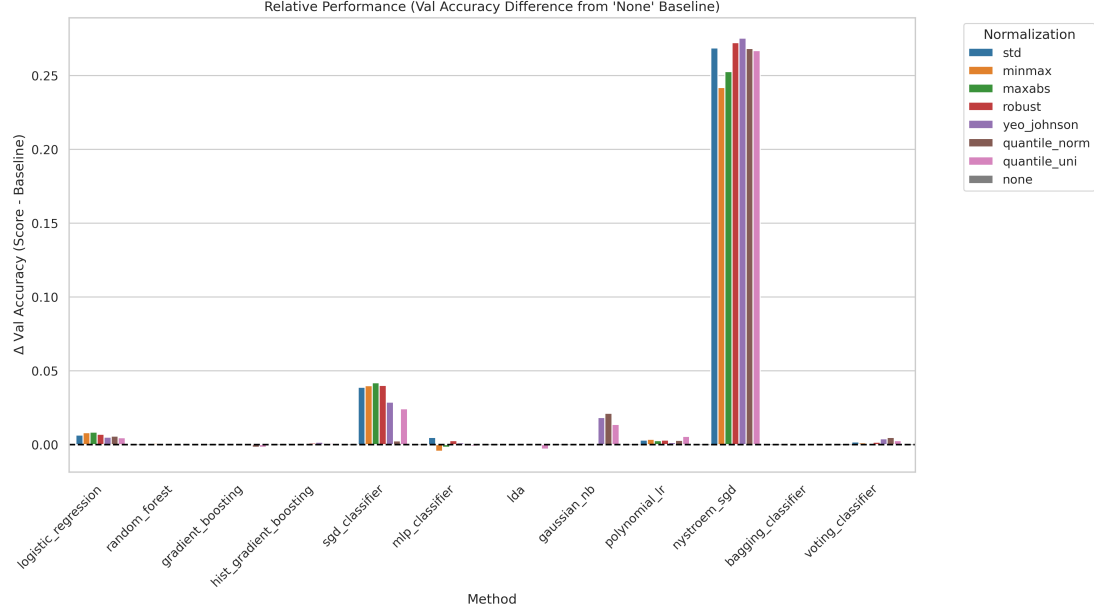


Fig. 6. This Figure shows the performance difference of data scaling methods in comparison to no scaling.

To get an overview of how data scaling methods affect model performance, we conducted an experiment comparing them. Figure 5 shows the results of this experiment.

While rescaling almost always performs better, it shows substantial improvements only in the case of Nystroem SGD. For some algorithms, there is no change at all, which can be explained by their internal structure. A good example of that is a decision tree, which always sets the same splits independent of rescaling. This observation is also shown in the relative comparison plot 6. When we compare the methods, we find that in most cases there is no measurable difference. Considering this, we decided to use the most commonly used Standard scaling for further experiments. This is not the optimal choice for all models, but it is very close. In addition, Standard scaling is very fast to compute, easy to understand and implement, and well known in the research community.

5.3 Data Normalization

In the following we will talk about data normalization. Note that normalization and scaling are often mixed and used as synonyms. In this section, we discuss per-sample normalization rather than per-feature rescaling, as we did in 5.2. Figure 7 and 8 show the absolute and relative accuracy performance of the three different normalization methods.

As the figures show, only the Gaussian Naive Bayes gains noticeable performances through this normalization. All other ML algorithms don't benefit at all, or only negligibly.

5.4 Feature Selection

The following section goes into a detailed ablation study on our feature set selection. Selecting the input features is a fundamental part of developing an ML model. Depending on the architecture, more input features can improve or worsen the model performance. To test the feature combination described in Table 2, we trained all architectures three times: Only basic features,

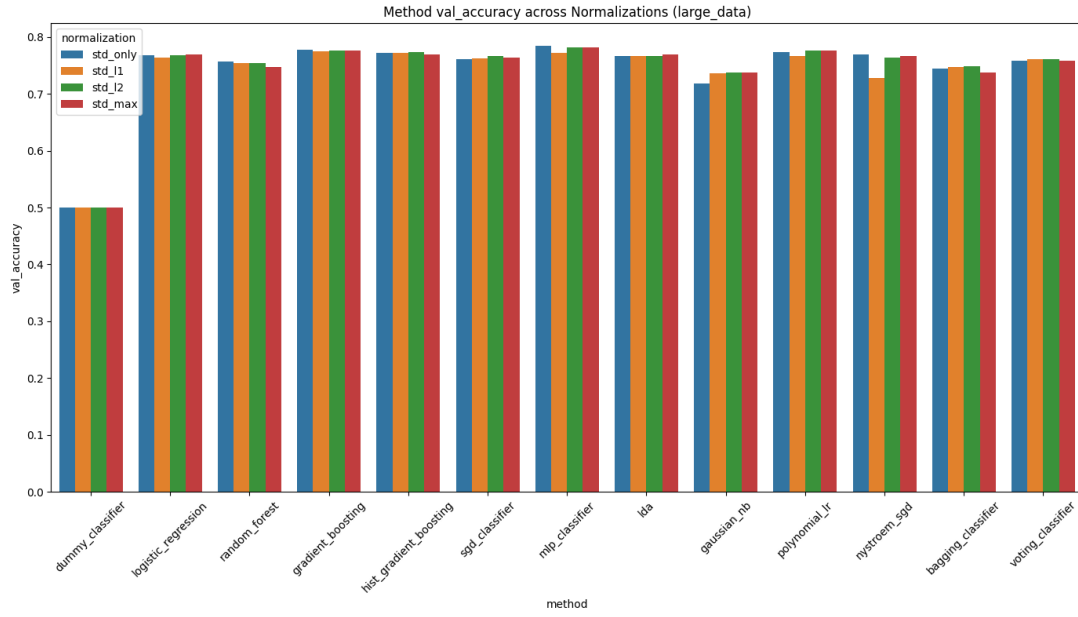


Fig. 7. The plot above shows different per-sample normalization method performances for 13 models. For a detailed visualization of the relative differences, refer to figure 8

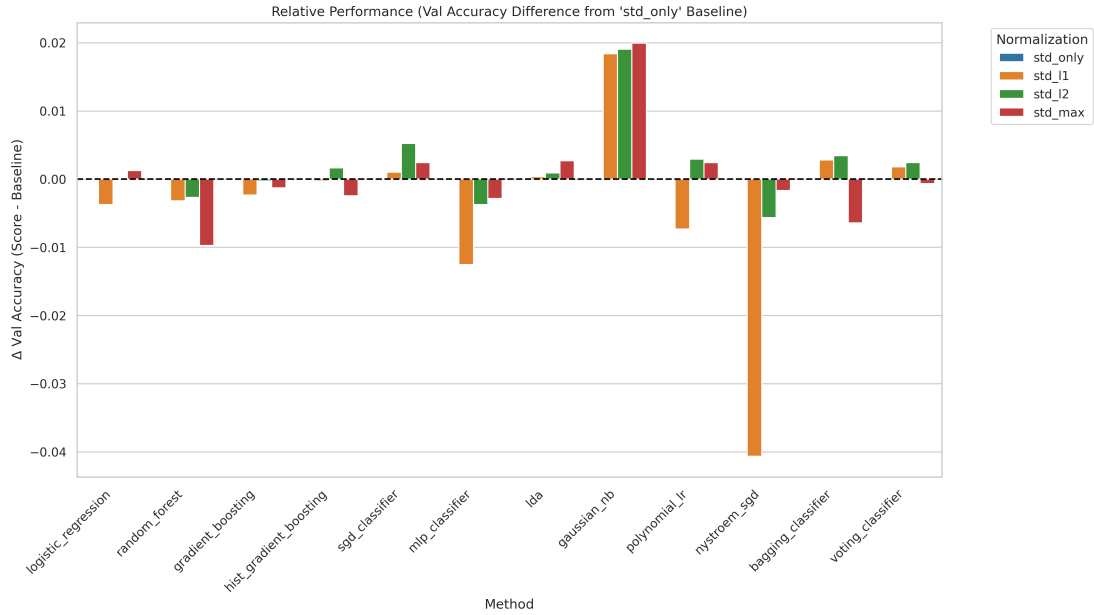


Fig. 8. This Figure shows the relative performance difference of the different per-sample normalization methods. As with data scaling, only the results for the Gaussian Naive Bayes benefit significantly from normalization. All other models perform worse, or the performance difference is indecisive. Refer to Figure 7 for the absolute results.

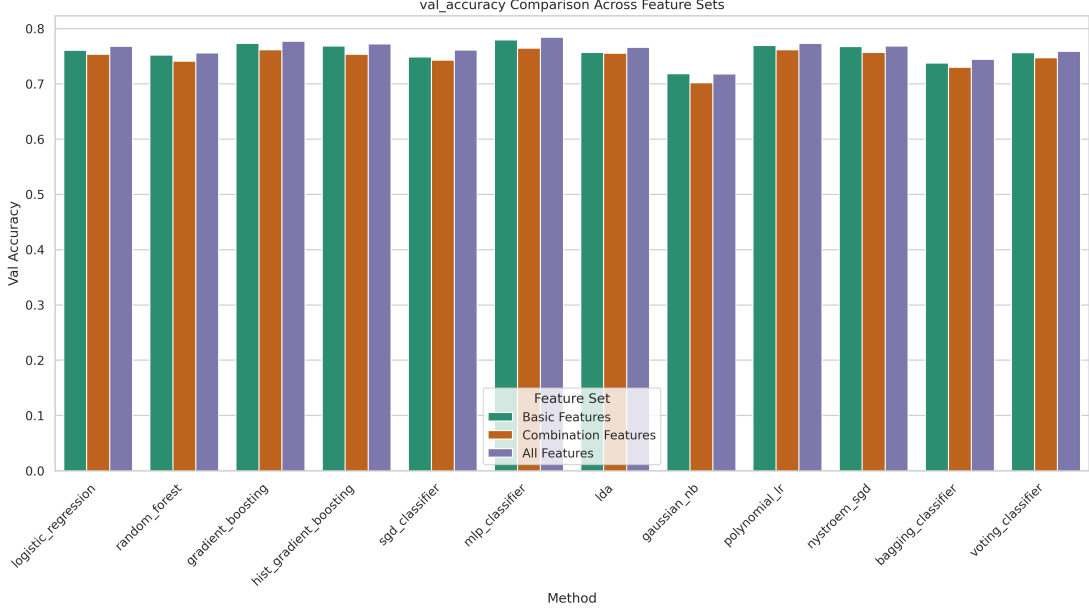


Fig. 9. This Figure shows the ablation study on the feature selection. It is clear that the new features incorporate almost all information of the basic features. In combination, they are able to slightly improve performance over the basic feature set. The experiment is based on standard scaling, no normalization and 3x oversampling. Figure 10 shows this plot in relative comparison to the basic feature performance.

only combination features and both. The result can be seen in figure 9. It becomes clear that most information is already contained in the basic feature set since all three set lead to very similar validation accuracy.

Looking further into the differences (Figure 10) it becomes clear that the combination features carry some extra information that is not already contained in the basic set. Depending on the architecture this can lead to improvements of up to $\hat{1.7}\%$ while bringing consistent improvements across all methods. Considering this, combination features provide a significant performance improvement in combination with the basic feature set.

5.5 Sampling Methods

As already discussed in subsection 4.3, sampling can be a useful tool to balance the training data. Since the benefit of different sampling methods heavily depends on the underlying algorithm architecture we conducted a detailed comparison for three sampling methods: undersampling, 3x oversampling and 10x oversampling. Note that no sampling at all is not a feasible baseline to compare against since most architectures overfit to the majority class without learning anything. The absolute and relative results in Figure 11 and 12 respectively, show that undersampling is performing consistently well while only being clearly outperformed in the SGD architecture. This is to some extent unexpected since architectures like MLPs should benefit in general by seeing more samples of $C_{\text{not } 1}$. The fact that this is not the case indicates that we already provide enough samples and more data does not bring any additional benefit.

5.6 Ensemble

A common approach to improving classification performance is to combine multiple models by having them vote. This is called ensemble and can significantly improve performance in cases

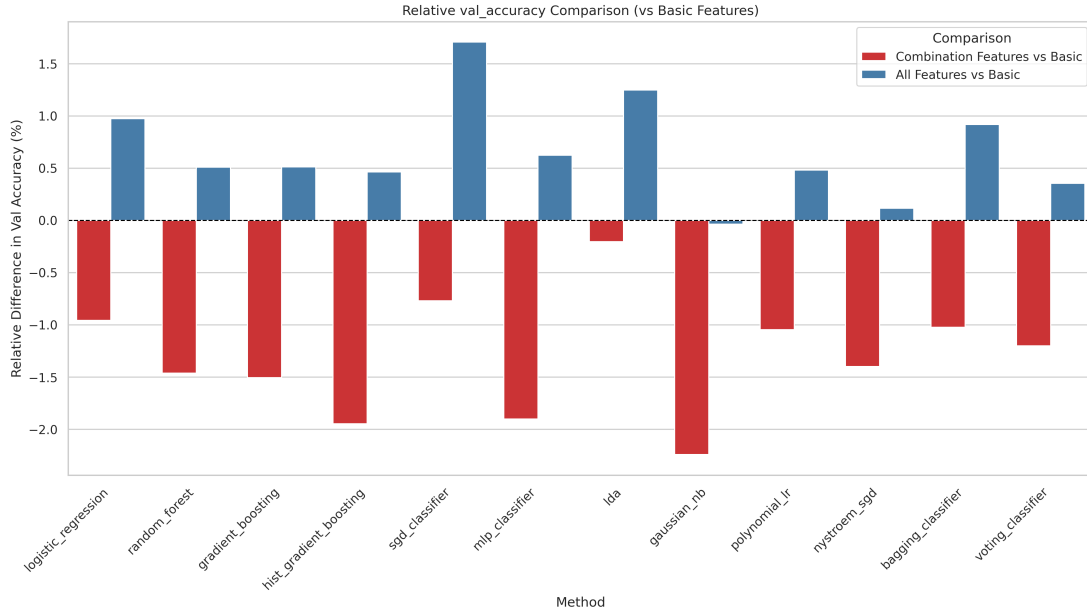


Fig. 10. This figure shows the relative difference for Figure 9. The experiment is based on standard scaling, no normalization and 3x oversampling.

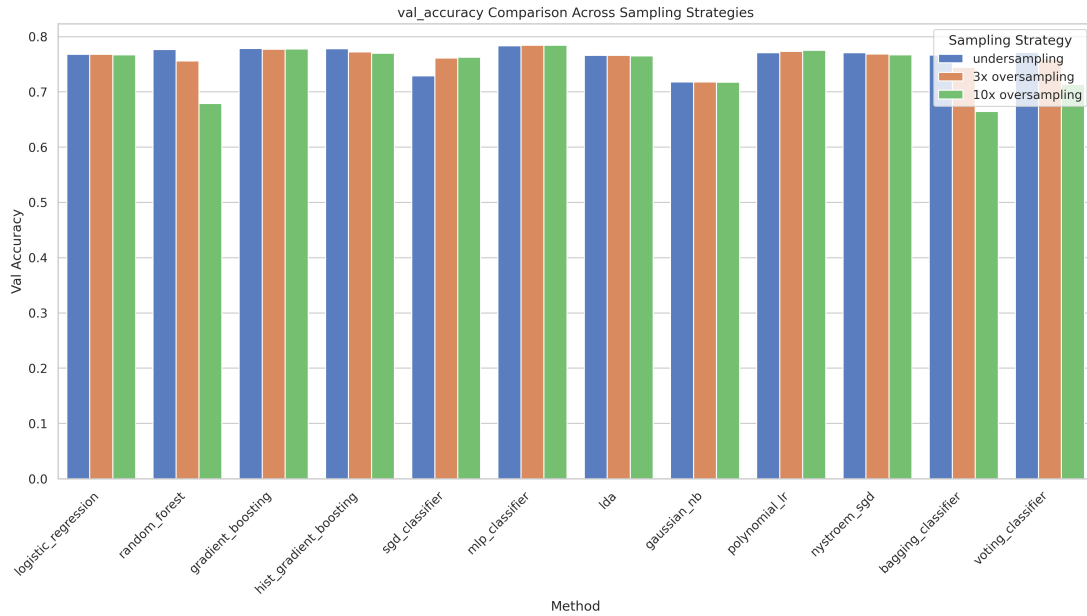


Fig. 11. This figure shows the accuracy results for different sampling methods. 3x and 10x stands for using a sample of C_1 three or ten times for every sample of $C_{\text{not } 1}$. More details about oversampling can be found in section 4.3.

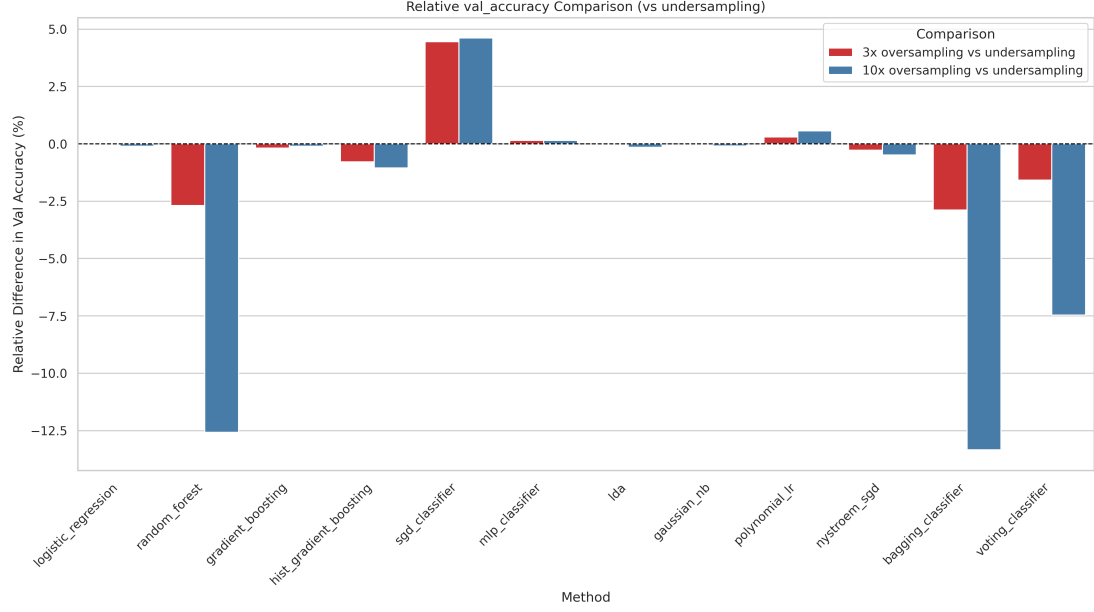


Fig. 12. For better analysis this plot provides the accuracy delta of 3x and 10x oversampling in comparison to undersampling. The absolute values can be found in Figure 11.

where the individual models have learned different indicators. To determine the effectiveness of such an ensemble we can look at Figure 13, which shows how many models classified a validation data point incorrectly. We can see that the models have a very high rate of agreement. This is clear when looking at the two extreme cases, which make up a large part of the total samples, meaning that often all models are correct, or all models are incorrect. This points to poor diversification and suggests that the models use similar clues in the data.

This bad diversification is also indicated by the large agreement rate of the models. Figure 14 shows the agreement of each model pair. Note that the agreement rate drops when one of the models performs poorly as demonstrated by the dummy classifier and Gaussian Naive Bias. Considering this, the rate of agreement is very large and confirms our observation regarding Figure 13.

While both Figures 13 and 14 indicate a low success rate for orchestration, it is still possible that a noteworthy model combination outperforms individual models. To manage such an orchestration of models, we need to choose the models we want to vote on the final classification. There are two approaches to choosing models. Either we take the best n models or the k most diverse ones. The intuition behind this is quite simple. We want the best models or the most diverse ones to vote. A middle ground would be to use the most diverse ones, but exclude the worst-performing models to avoid unqualified opinions.

As indicated by Figure 15, no combination of model orchestration yields an improvement over the best-performing individual model. As described earlier, this is expected because of the high agreement rate between the models.

An intuitive example to further understand this is: We conduct an experiment with 10 people. All of them are tasked to tell the brand of a car. While all the 10 people are not experts, they only rely on the logo at the front of the car to determine the brand. This works in general but in cases where the front logo is covered, all of them fail. This means they all rely on the same feature for their prediction rather than diversifying. If we now ask them as a group, they will perform equally well or badly because they still only rely on the front logo.

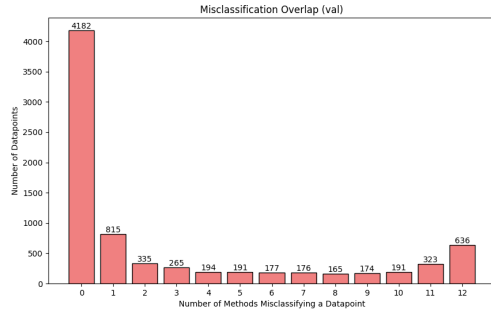


Fig. 13. This Figure shows the number of models that misclassified a specific sample. Meaning that 4182 validation samples were correctly classified by all 12 models (not including the dummy classifier). Furthermore, 815 validation data points were classified incorrectly by one model, and so on.

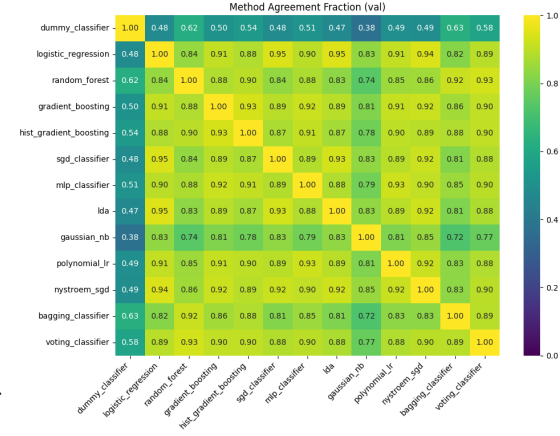


Fig. 14. This Figure shows the agreement rate of all binary model pairs. The value can be interpreted as chance that the two models classify a sample similarly. Example: mlp_classifier and gradient.boosting have a value of 0.92, meaning that they classify samples 92% of the time the same.

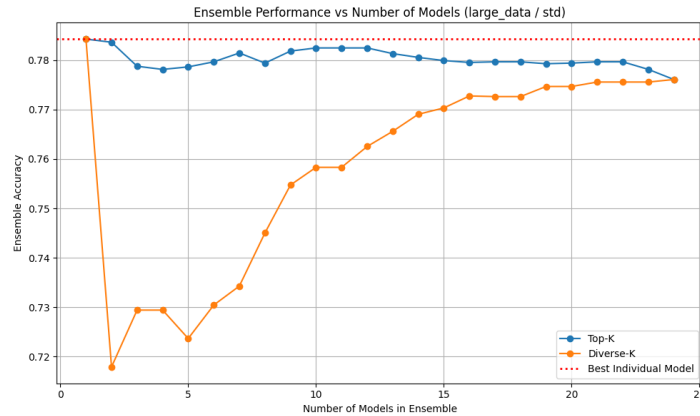


Fig. 15. This Figure shows the Ensemble performance for different model combinations. The dotted red line indicates the best individual model performance and, therefore, the performance to beat. The orange datapoints indicate the performance of the k most diverse models based on agreement rate. In contrast, the blue data points indicate the performance of the best k models in an ensemble. The pool of models contains the 12 models previously discussed (excluding the dummy classifier). For all models, we train one 3x oversampled version and one undersampled version.

This is also the case in our experiment. As all models seem to rely on the same features, their group vote does not yield any performance increase.

5.7 DeepSet

Experimental Results We evaluated DeepSets with different sized sets on the binary C_1 versus $C_{\text{not } 1}$ classification task. As shown in Table 3, increasing the number of events in the set leads to higher accuracy. This shows that having more then a single event will give more clarity on the cvv value. This is intuitional using more events as input will increase the likelihood of having a event that sits on the border of the distribution, which makes discrimination easier.

Table 3. Results for evaluating different set sizes on deep-set.

Set-Size	Accuracy	Precision	Recall
1	77.58%	77.57%	77.56%
2	86.09%	85.22%	85.21%
3	90.27%	90.48%	90.47%
5	95.51%	95.28%	95.28%
10	99.11%	98.82%	98.82%

Also in the Recall and Precision values it is seen that the model does not tend into one direction of the binary classification, but is balanced across both classes.

Ablition At the set level (aggregating multiple events), we tested four aggregation approaches:

- **Self-attention aggregation:** The encoded events are processed through self-attention layers before aggregation, allowing the model to learn pairwise interactions between events in the set.
- **No self-attention:** A baseline where self attention layers are removed.
- **Attention pooling:** A learned query vector attends to all events, producing a weighted sum that serves as the aggregated representation. This is sometimes referred to as weighted mean pooling with attention weights.
- **Max aggregation instead of attention pooling:** A simple permutation-invariant operation that takes the element-wise maximum across all encoded events.

Our ablation study on self-attention (Figure 16) revealed that incorporating self-attention and attention pooling into our model does not provide significant improvements. This means that the model was not able to leverage connections between event features and also the simple max aggregation is enough to capture full information on the events.

5.8 Best Results

Based on all the ablation studies we presented earlier, we were able to find the best constellation of settings for each ML algorithm. As Figure 17 shows, there is not a single configuration that fits all ML architectures. But we can see that if configured correctly most methods get really close to the best performing MLP. This suggests that the underlying data has a massive ambiguity problem. In the end, the deep learning based MLP architecture was able to outperform all other architectures. It provides a 0.78% lead over the second-best Gradient Boosting Classifier. You can find more details about the best performing MLP in the Appendix D.

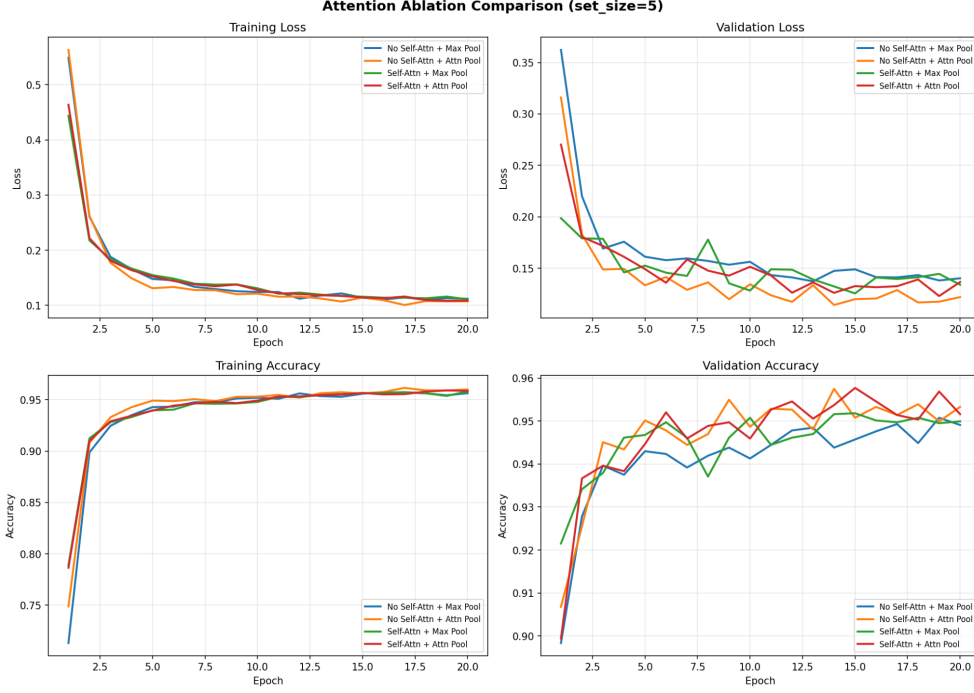


Fig. 16. Ablation study of DeepSet testing out the impact of self-attention and attention pooling on the performance of the classification. Validation accuracy shows that all different combination will result in similar performance. Not showing a positive impact of the attention mechanism in our experiment.

6 Conclusion and Outlook

This project studied the binary classification of VBF Higgs-pair production events in the semileptonic channel $HH \rightarrow bbq\ell\nu$, with the goal of determining whether a matched, truth-level event representation contains enough information to distinguish between the Standard Model coupling point at $\text{cvv} = 1$ from all other cvv values.

The dataset construction revealed a class imbalance. While samples at other coupling points are matched at rates of 10 – 21%, the Standard Model samples survives selection and jet matching at only 0.4%. Because the Standard Model sample is so strongly depleted after matching, separating C_1 from $C_{\text{not},1}$ is a genuinely difficult task. In terms of both data statistics and physical relevance.

The features we used to describe each event were chosen to match the physical structure of the collision. This means we included information about the VBF quarks, both W decays, and the $H \rightarrow b\bar{b}$ system separately, as well as combined quantities built from multiple objects together. Looking at how the features correlate with each other confirms this was a good choice. Features that describe the same physical system are most strongly related to each other, which is exactly what you would expect. Adding the combination features on top of the basic ones gave small but consistent accuracy improvements of up to 1.7% across all models.

The preprocessing experiments gave clear practical takeaways. Standard scaling works well across all models and is a safe default choice. Normalizing individual samples (per-sample normalization) gave no meaningful benefit for most models. Using $3\times$ oversampling for the minority class was a good middle ground. It avoids throwing away too much data (undersampling) without repeating the same minority samples too many times (aggressive oversampling).

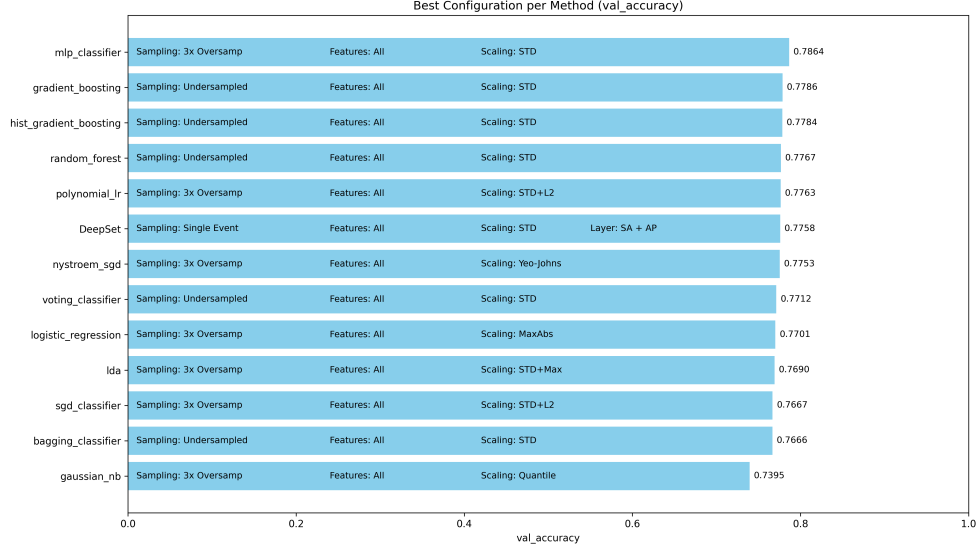


Fig. 17. This Figure shows the best performance of all methods considering all possible combinations from the ablation studies. Note that the DeepSet was designed based on other configuration parameters as the other methods. The DeepSet uses self attention (SA) and attention pooling (AP). For more details refer to section 5.7.

When all models are tuned to their best settings, they end up performing at surprisingly similar accuracy levels. This suggests the difficulty lies in the data itself rather than in which model you choose. The data simply does not contain enough clear signal to let any model pull far ahead. Ensemble voting confirms this: because all models tend to agree on the same predictions, both correct and incorrect. Combining them does not improve over the best single model. They all appear to rely on the same features.

The MLP achieved the highest accuracy among models that classify one event at a time, outperforming the second-best method, Histogram Gradient Boosting Tree, by 0.78%. DeepSets takes a different approach by classifying sets of multiple events at once rather than single events. When given ten events at a time, it reaches 99.1% accuracy, compared to 77.6% with just one event. This improvement comes with an important caveat: the model has access to more information per prediction, so the comparison is not directly fair to the single-event models. Still, the results show that the coupling signal becomes much clearer when multiple events are considered together. Interestingly, adding self-attention or attention pooling inside the DeepSets model did not improve results, meaning a simple aggregation was already enough to capture the relevant information.

There are some limitations to keep in mind. All results are based on truth-level simulation, without any detector effects. No separate test set was used, so the reported numbers come from the validation split only. Future work could apply the same pipeline to detector-level data, use larger datasets, or test whether the strong DeepSets results hold under more realistic conditions. Nevertheless, since VBF Higgs-pair production is predicted by the Standard Model but has not yet been observed, and the available event rates are extremely low, the fact that the methods developed here achieve strong classification performance shows that ML-based approaches can provide meaningful discrimination for this challenging process. This study provides a clear and reproducible baseline for coupling classification in this channel, and the constructed dataset and framework offer a solid foundation for future experimental analyses targeting the κ_{2V} coupling.

A Feature Correlation

Feature correlation is an important tool to understand how features interact with each other. Figure 18 shows a structured feature correlation matrix. It shows which features correlate positively (red) or negatively (blue). We can use this information to select features that strongly correlate with our target class or read it as an indicator for redundant information. Figure 18 shows that similar properties of different particles strongly correlate.

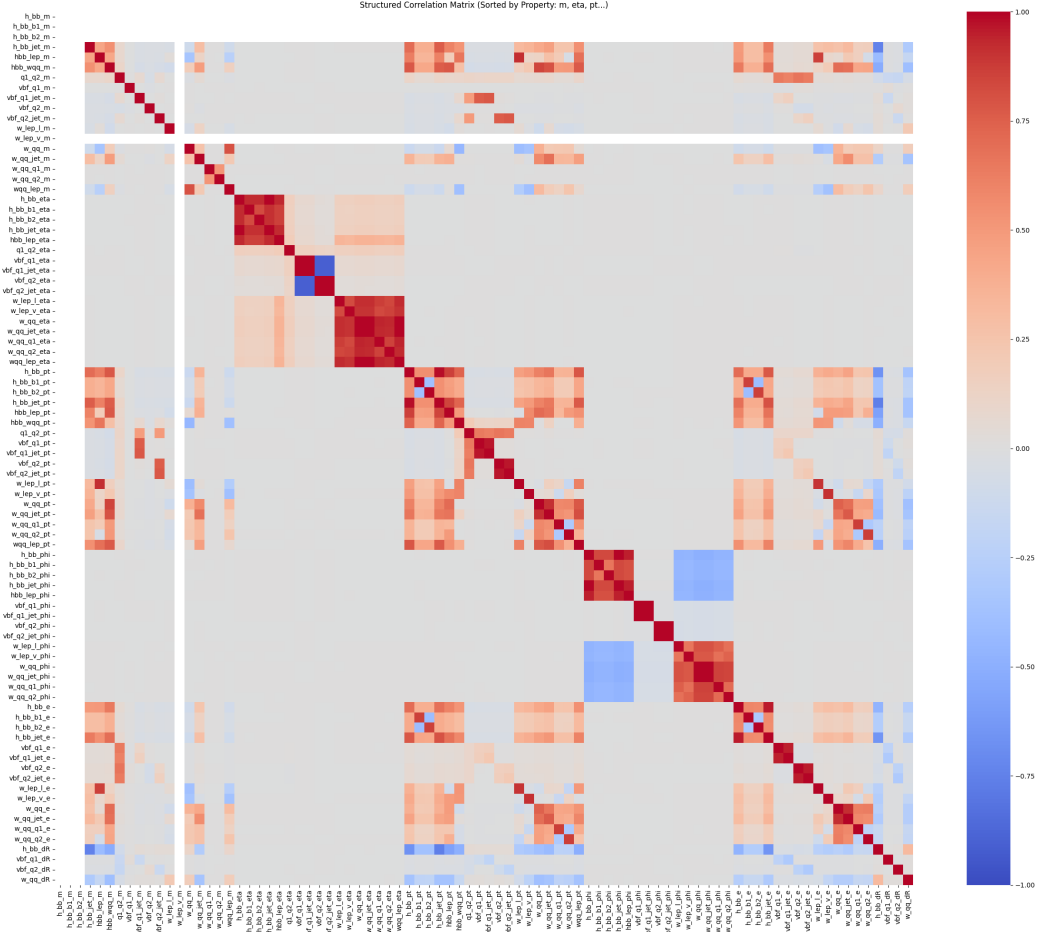


Fig. 18. Structured correlation matrix of the exported event features. The variables are sorted by kinematic type, which makes block-like correlations between related physical systems visible. A simplified interpretation of the values is: 0.2 = *bad*, 0.5 = *ok*, 0.8 = *good*, 1.0 = *perfect correlation*.

B Framework Configuration

You can find an example configuration file in the following. This configures the experimental run based on one data source and two classes. Running this experiment trains 13 models for 7 scaling methods which totals in 91 combinations. For more details on the individual fields refer to Table 4.


```

{
  "experiment_name": "3x_large_all_features_with_scaling_no_norm",
  "datasource": [
    {
      "name": "large_data",
      "loader": "standard_loader",
      "data_path": "ml_framework/data/large/matched_events_all_no_dup.csv",
      "val_split": 0.2,
      "balance_val": true,
      "target_column": "cvv",
      "drop_columns": ["event_id", "vbf_q1_phi", "vbf_q1_e", "vbf_q1_m",
        "vbf_q1_jet_phi", "vbf_q1_jet_e", "vbf_q1_dR", "vbf_q2_phi",
        "vbf_q2_e", "vbf_q2_m", "vbf_q2_jet_phi", "vbf_q2_jet_e",
        "vbf_q2_dR", "w_qq_phi", "w_qq_m", "w_qq_jet_phi", "w_qq_dR",
        "w_qq_q1_eta", "w_qq_q1_phi", "w_qq_q2_eta", "w_qq_q2_phi",
        "w_lep_l_phi", "w_lep_v_phi", "h_bb_eta", "h_bb_phi", "h_bb_m",
        "h_bb_jet_eta", "h_bb_jet_phi", "h_bb_dR", "h_bb_b1_eta",
        "h_bb_b1_phi", "h_bb_b1_m", "h_bb_b2_eta", "h_bb_b2_phi",
        "h_bb_b2_m"],
      "merge_classes": [[0.0, 0.5, 1.5, 2.0, 3.0]],
      "discard_classes": [],
      "balance_train": "oversample",
      "oversample_factor": 3.0
    }
  ],
  "normalization": [
    {
      "name": "std",
      "scaling": "standard"
    },
    {
      "name": "minmax",
      "scaling": "minmax"
    },
    {
      "name": "maxabs",
      "scaling": "maxabs"
    },
    {
      "name": "robust",
      "scaling": "robust"
    },
    {
      "name": "yeo_johnson",
      "scaling": "yeo-johnson"
    },
    {
      "name": "quantile_norm",
      "scaling": "quantile_normal"
    }
  ]
}

```

```

        "name": "quantile_uni",
        "scaling": "quantile_uniform"
    }
],
"save_models": true,
"method_n_jobs": 5,
"methods": [
    "dummy_classifier",
    "logistic_regression",
    "random_forest",
    "gradient_boosting",
    "hist_gradient_boosting",
    "sgd_classifier",
    "mlp_classifier",
    "lda",
    "gaussian_nb",
    "polynomial_lr",
    "nystroem_sgd",
    "bagging_classifier",
    "voting_classifier"
],
"output": {
    "dir": "results/ablation/"
}
}

```

C Detailed 5 Class Classification

To motivate our decision to focus only on C_1 and C_{not_1} classification, we created Figure 19. There, you can see the performance when classifying all classes ($C_v \in [0, 0.5, 1.5, 2.0, 3.0]$) available in the dataset except for C_1 . The results show that all models struggle to significantly exceed random guessing. This indicates that the input distributions of all classes are almost identical and can not be distinguished. For this reason, we decided to focus on classifying C_1 and C_{not_1} .

D Best Performing MLP

D.1 Architecture

The deployed Multi-Layer Perceptron (MLP) architecture relies on a progressively shrinking funnel topology ($256 \rightarrow 128 \rightarrow 64$ nodes, also refer to Figure 20). This architectural choice is explicitly motivated by the high dimensionality of the full "all features" dataset (containing all low-level spatial and kinematic interactions). It provides sufficient parameter density to learn highly non-linear, multi-dimensional decision boundaries while systematically compressing the intermediate representations.

D.2 Setup

The combination of data augmentation (3x oversampling) and standard scaling (std) forms a critical backbone for this model:

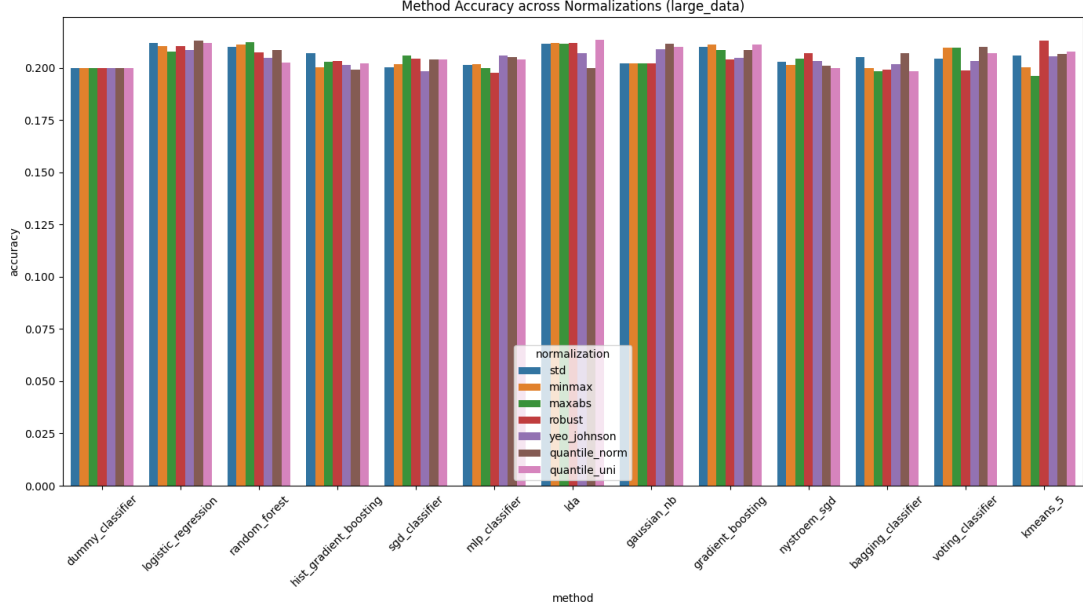


Fig. 19. This shows the results for the five classes C_v with $v \in [0, 0.5, 1.5, 2.0, 3.0]$. As the dummy classifier suggests, the expected performance of random guessing is 20%.

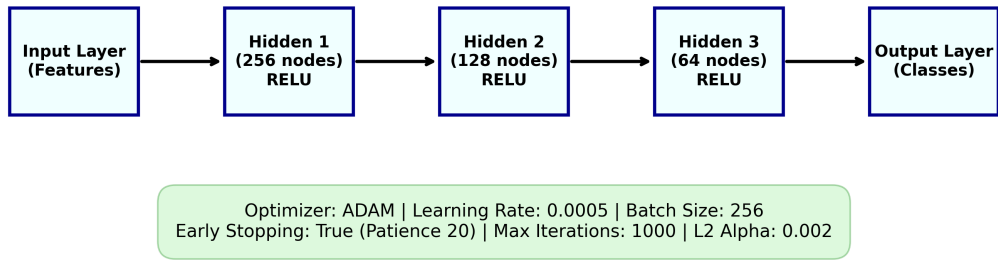


Fig. 20. This figure show the structure and training parameters of the best performing MLP model. The number of input features and therefore the number of input neurons are 53 while having one output neuron per class (two in total).

Feature Scaling (std): Neural networks trained with gradient descent, such as adam, are highly sensitive to physical features measured on wildly disparate scales (e.g., coordinates η versus huge momenta p_T). Standardizing inputs feature-wise to yield a zero mean ($\mu = 0$) and unit variance ($\sigma^2 = 1$) ensures that gradients behave stably and symmetrically, allowing the ReLU activation functions ($\max(0, x)$) to consistently avoid unrecoverably saturated regimes during early training epochs.

L2 Regularization ($\alpha = 0.002$) & Early Stopping: Since 3x oversampling exposes the network to highly repetitive synthetic representations of the minority signal class C_1 , the model runs a substantial risk of overfitting (memorization). By combining a rigid L^2 weight decay penalty with strict early stopping criteria, the architecture enforces robust generalized learning while aggressively truncating parameter refinement whenever validation generalization ceases.

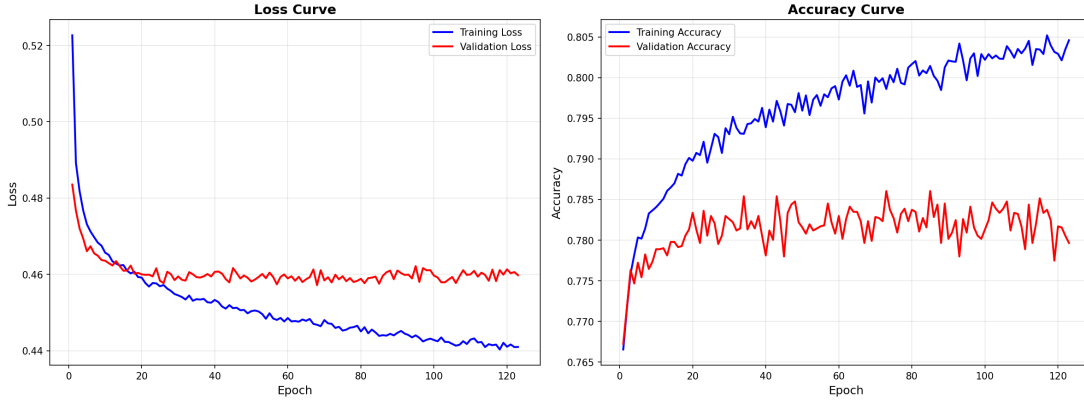


Fig. 21. This figure shows the MLP training curves and scores. The curves suggest a slight overfitting problem. For visualization reasons the early stopping was delayed in this chart.

D.3 Results:

Based on the validation dataset results, the network proved highly effective, achieving an aggregate overall accuracy of 78%. The model maintains impressively balanced performance across both classification categories, yielding matching global F_1 -scores of 0.78 built up from closely aligned precision (0.79 vs 0.78) and recall (0.78 vs 0.79) measurements.

The confusion matrix (Figure 22) confirms that the 3x oversampling strategy was profoundly successful in mitigating underlying background imbalances natively inherent in scattering topologies. The network acts neutrally and almost completely unbiased when resolving the true categories, generating a cleanly symmetric diagonal space that isolates false positives at almost exactly the same magnitude as false negatives. Or in simpler terms, the MLP is not biased towards any of the two classes.

We can look further into the failure cases of the MLP model by plotting the misclassifications against individual features. An example for the feature w_{qq_e} can be found in Figure 23. As we can see in this case but also for all other features, the mistakes are seemingly random distributed across the feature range. This is exactly what we expect of a well-trained model and demonstrates that there are obvious and no easy-to-learn tells in the data left.

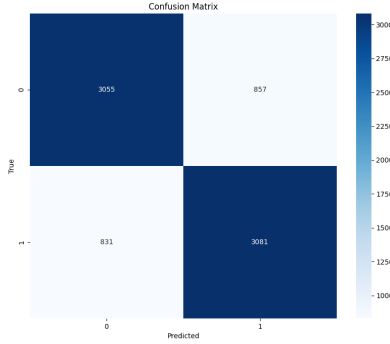


Fig. 22. Confusion Matrix of the best performing MLP.

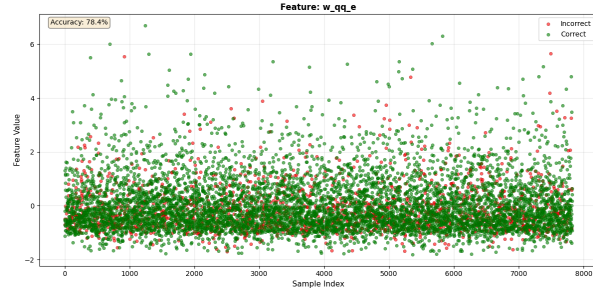


Fig. 23. Misclassification graph for feature "w_qq_e". This shows that the misclassifications are randomly distributed over the feature.

E Feature Distributions

Figures 24–28 show the per-feature distributions for C_1 ($\text{cvv}=1$) and $C_{\text{not } 1}$, grouped by kinematic property. The energy distributions (Figure 24) show strong overlap for nearly all variables, confirming that energy-scale information alone provides limited discriminating power. The pseudorapidity distributions (Figure 25) show more visible shape differences, particularly for VBF-side variables, where C_1 events tend to populate a broader and more central range. The mass distributions (Figure 26) are dominated by sharp peaks at the known particle masses for truth-level objects, while jet-level and combination masses show broader shapes with moderate class separation. The azimuthal angle distributions (Figure 27) are nearly uniform for both classes, as expected from the rotational symmetry of the detector, and carry essentially no separation power. The ΔR matching distances (Figure 28) show narrow, strongly peaked distributions with only minor shape differences between classes.

F Codebase

The codebase for reproducing results and running further experiments is publicly available in two repositories:

The machine learning section can be found [here](#)

The DeepSet related code can be found [here](#).

References

1. Bottou, L.: Large-scale machine learning with stochastic gradient descent. In: Lechevallier, Y., Saporta, G. (eds.) *Proceedings of COMPSTAT'2010*. pp. 177–186. Physica-Verlag HD, Heidelberg (2010)
2. Breiman, L., Friedman, J., Stone, C., Olshen, R.: *Classification and Regression Trees*. Taylor & Francis (1984)
3. Breiman, L.: Random forests. *Machine Learning* **45**(1), 5–32 (2001). <https://doi.org/10.1023/A:1010933404324>
4. Cox, D.R.: The regression analysis of binary sequences. *Journal of the Royal Statistical Society. Series B (Methodological)* **20**(2), 215–242 (1958)
5. Friedman, J.H.: Greedy function approximation: A gradient boosting machine. *Ann. Statist.* **29**(5), 1189–1232 (October 2001). <https://doi.org/10.1214/aos/1013203451>

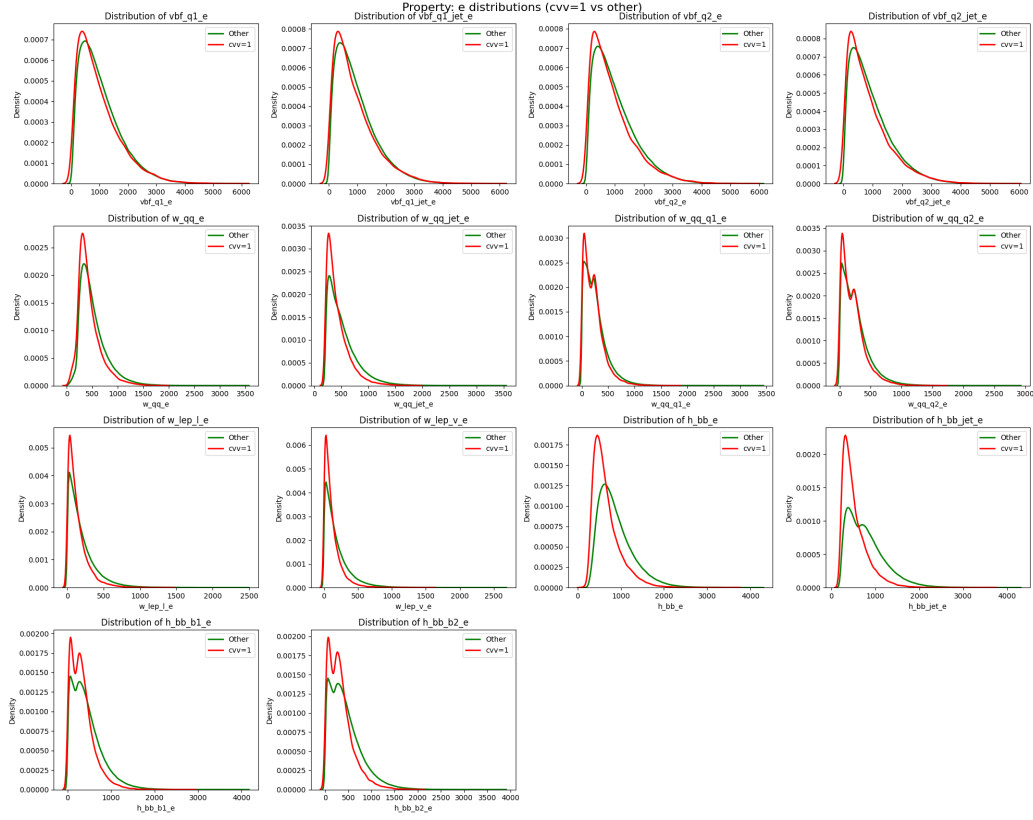


Fig. 24. Distributions of all energy-related features for C_1 (red) and $C_{\text{not } 1}$ (green). Most variables show strong overlap between the two classes.

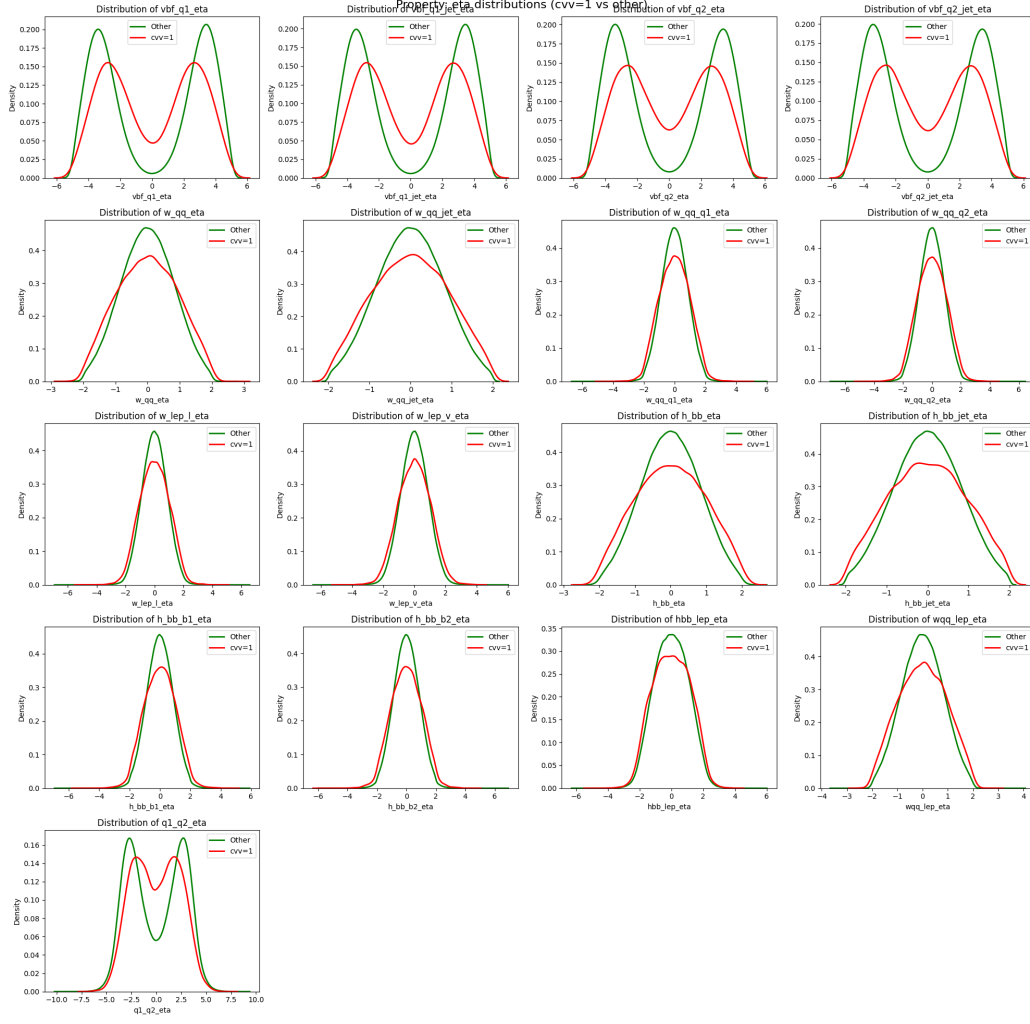


Fig. 25. Distributions of all pseudorapidity-related features for C_1 (red) and $C_{\text{not } 1}$ (green). Several VBF-side variables show visible shape differences between the two classes.

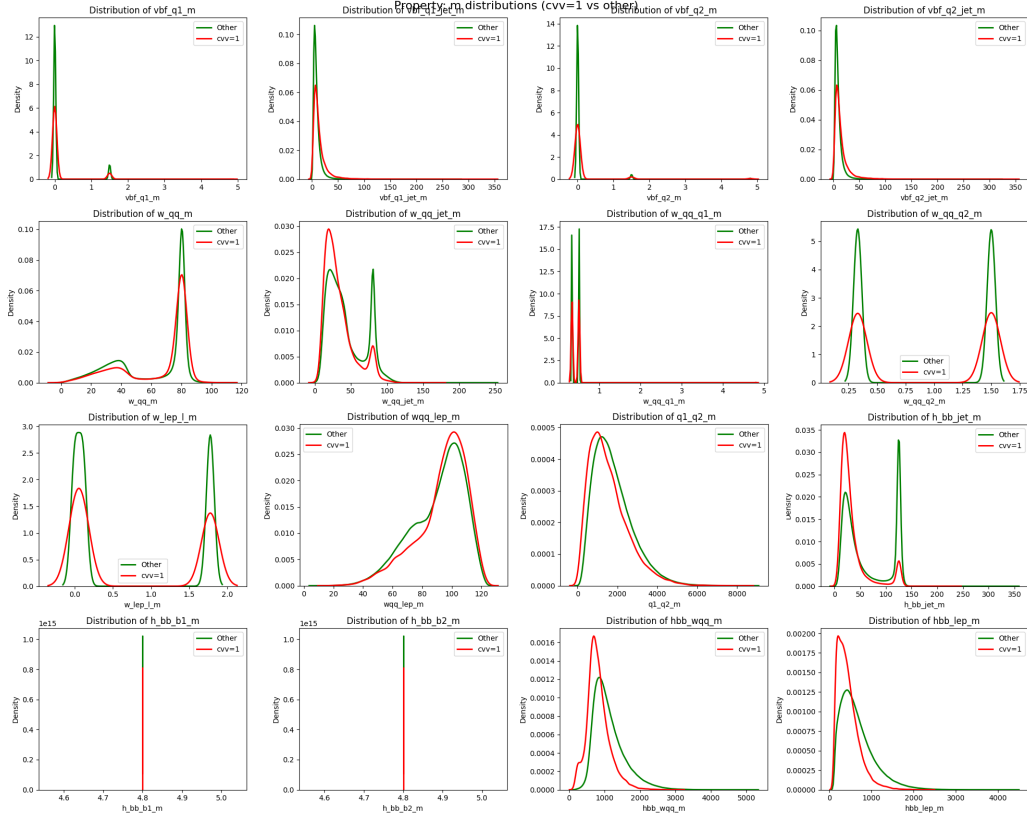


Fig. 26. Distributions of all mass-related features for C_1 (red) and $C_{\text{not } 1}$ (green). Truth-level masses show sharp peaks at known particle masses, while jet-level and combination masses exhibit broader distributions with moderate separation.

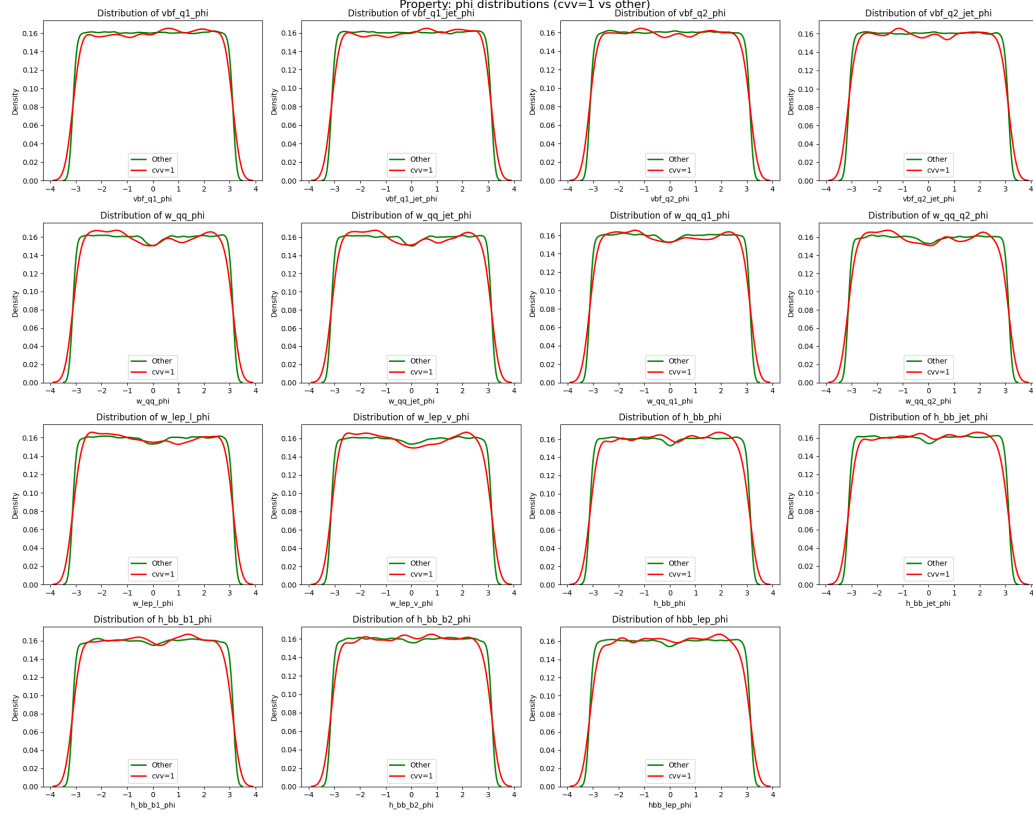


Fig. 27. Distributions of all azimuthal angle features for C_1 (red) and $C_{\text{not } 1}$ (green). The distributions are nearly uniform for both classes, consistent with the rotational symmetry of the detector geometry.

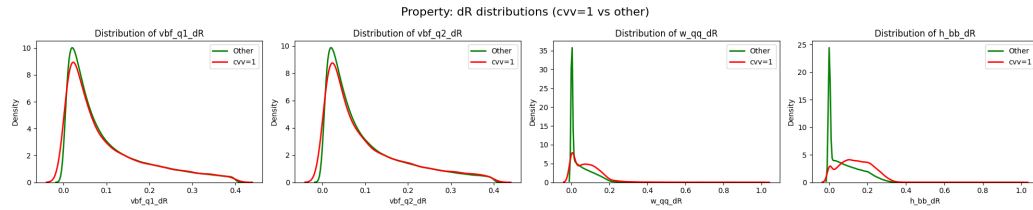


Fig. 28. Distributions of all ΔR matching distance features for C_1 (red) and $C_{\text{not } 1}$ (green). The distributions are narrow and strongly peaked, with only minor differences between the two classes.

6. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016), <http://www.deeplearningbook.org>
7. Navas, S., et al.: Review of particle physics. Phys. Rev. D **110**, 030001 (2024). <https://doi.org/10.1103/PhysRevD.110.030001>
8. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É.: scikit-learn: Machine learning in python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
9. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need (2023), <https://arxiv.org/abs/1706.03762>
10. Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., Smola, A.: Deep sets (2018), <https://arxiv.org/abs/1703.06114>

Table 4. Comprehensive Description of Framework JSON Configuration Fields

Field	Data Type	Description
experiment_name	String	The unique identifier for the experiment. Dictates the root folder name inside the output directory where all results are saved.
datasource	Array	Defines one or more datasets to evaluate. The framework independently loops over each data source.
– name	String	A descriptive alias for the specific dataset permutation (used in output path).
– loader	String	The registered dataloader class to instantiate (e.g., standard_loader).
– data_path	String	The absolute or relative file path to the single combined CSV dataset (if utilizing dynamic runtime splitting).
– train_path	String	Path to a statically defined training CSV (used if data_path is omitted).
– val_path	String	Path to a statically defined validation CSV (used if data_path is omitted).
– test_path	String	Path to a statically defined hold-out test CSV.
– val_split	Float	The mathematical ratio (e.g., 0.2 for 20%) of data_path reserved for validation.
– balance_val	Boolean	If true , forces class distribution balancing (via under-sampling) specifically on the validation splits. Defaults to true .
– target_column	String	The explicit dataframe column name representing the dependent/target variable (e.g., cvv).
– drop_columns	Array	A list of feature names to aggressively discard from the dataset prior to training.
– merge_classes	Array	Rules for physically merging subset labels into macro classes before classification (e.g., [[0.0, 0.5, 1.5]]).
– discard_classes	Array	Specific target labels to systematically filter and remove entirely from the dataset.
– balance_train	String	Mechanism for runtime data balancing uniquely affecting the training set (e.g., oversample , undersample).
– oversample_factor	Float	Scaling multiplier applied during oversampling to strictly cap artificial duplication relative to the minority class.
normalization	Array	Defines different preprocessing pipelines. The framework loops over each pipeline for every datasource.
– name	String	Alias for the preprocessing step (used for subdirectory scaling).
– scaling	String	Determines the global column-wise scaling approach. Supported values: standard , minmax , maxabs , robust , yeo-johnson , quantile_normal , quantile_uniform .
– normalization	String	Determines the row-wise (sample-wise) vector normalization norm. Supported values: l1 , l2 , max .
save_models	Boolean	Flag dictating whether the finalized models are permanently serialized (.pkl, .onnx) to disk.
method_n_jobs	Integer	Global threading parameter governing parallel CPU-thread allocation within models supporting parallel execution.
methods	Array	Defines the exact registered classification modeling classes (e.g., random_forest , mlp_classifier) to execute in the run.
output	Object	Output logistics configuration block governing artifact storage.
– dir	String	Root path representing the parent directory where the experiment folder structure is generated.